React & React Native

200 Interview Questions & Answers

A complete study guide — from fundamentals to senior-level architecture

★Questions marked with a star are the highest-priority, must-know concepts most likely to come up in real interviews.

Table of Contents

1. React & React Native Fundamentals

Core concepts underpinning React and React Native.

Q1. ★What is React, and what problem does it solve?

A: React is a JavaScript library for building user interfaces out of small, reusable components. It solves the problem of manually keeping the UI in sync with changing data: instead of writing imperative DOM-manipulation code, you describe what the UI should look like for a given state, and React efficiently updates the screen when that state changes.

Q2. ★What is React Native, and how does it differ from React for the web?

A: React Native is a framework that uses React's component model and JavaScript to build native mobile apps for iOS and Android. Unlike React DOM, which renders HTML elements in a browser, React Native renders actual native UI components (UIView, android.view) through a native bridge/JSI, so apps look and perform like native apps rather than web pages wrapped in a WebView.

Q3. ★How does React Native render native views instead of HTML/DOM elements?

A: React Native components like <View> and <Text> are JavaScript wrappers around native platform widgets. React's reconciler produces a tree of native UI instructions which are sent across the bridge (or directly via JSI in the New Architecture) to the native side, where the platform creates and updates real native views instead of DOM nodes.

Q4. ★What is the Virtual DOM, and how does React use it to update the UI efficiently?

A: The Virtual DOM is a lightweight in-memory representation of the UI tree. When state changes, React builds a new virtual tree, diffs it against the previous one (reconciliation), and computes the minimal set of real DOM (or native view) mutations needed, applying only those changes instead of re-rendering everything from scratch.

Q5. ★What is the New Architecture (Fabric + TurboModules + JSI), and what problem does it solve compared to the old bridge?

A: The New Architecture replaces the old asynchronous, JSON-serializing bridge with JSI (JavaScript Interface), which lets JS and native code call each other directly and synchronously when needed. Fabric is the new rendering system built on JSI for faster, more consistent layout/rendering, and TurboModules lazily load native modules on demand, together reducing latency, serialization overhead, and startup cost.

Q6. ★What is reconciliation, and how does React decide what to re-render?

A: Reconciliation is the algorithm React uses to compare a new element tree with the previous one to determine the minimal set of changes needed. It uses heuristics such as comparing element types at each position and using `key` props on list items to match elements across renders, avoiding a full $O(n^3)$ tree comparison.

Q7. ★What are keys in React, and why are they important in lists?

A: Keys are stable, unique identifiers React uses to match array items between renders during reconciliation. Without proper keys, React may misidentify which items changed, added, or removed, causing incorrect DOM/native state (like wrong input values) to persist on the wrong element and hurting performance.

Q8. ★What is a 'controlled' vs 'uncontrolled' component?

A: A controlled component has its value driven entirely by React state — the component's input is set via props and changes are handled through an onChange-style callback that updates state. An uncontrolled component manages its own internal state (e.g. via a ref) and React doesn't dictate its value on every render.

Q9. What are the core building-block components in React Native (View, Text, Image, ScrollView)?

A: View is the basic container (like a div) supporting Flexbox layout; Text renders and styles text and is the only component that can contain raw strings; Image displays images from local or remote sources; ScrollView renders all its children at once inside a scrollable container, useful for small, non-virtualized content.

Q10. Why can't you use HTML tags like or in React Native?

A: React Native doesn't render to a browser DOM, so there's no HTML parser or CSS engine involved — there is no to render. Instead it exposes its own native-backed component set (View, Text, etc.) that map to platform-native UI elements.

Q11. What is the role of Metro bundler in a React Native project?

A: Metro is React Native's JavaScript bundler. It resolves the module graph, transforms JS/TS/JSX (and assets) via Babel, bundles everything into a single JS file the app loads at runtime, and powers Fast Refresh during development by watching files and sending incremental updates.

Q12. What is the difference between React Native and frameworks like Flutter or native (Swift/Kotlin) development?

A: React Native renders true native UI components using JavaScript/React and shares a large amount of business logic across platforms, while Flutter renders its own widgets via its own Skia/Impeller-based rendering engine rather than native platform widgets. Fully native development (Swift/Kotlin) offers maximum control and performance per platform but requires writing and maintaining separate codebases for iOS and Android.

Q13. What is Expo, and how does it differ from a bare React Native project?

A: Expo is a toolchain and set of managed services built on top of React Native that simplifies setup, builds, OTA updates, and access to common native APIs without writing native code. A bare React Native project gives full control over native iOS/Android code but requires managing native tooling (Xcode/Gradle) directly; Expo's 'dev client' and prebuild workflow narrow this gap by allowing custom native modules within a managed-feeling workflow.

Q14. ★What is reconciliation, and how does React decide what to re-render?

A: See Q6 — reconciliation is React's diffing algorithm that compares the new and old element trees using type and key matching to compute the minimal update, re-rendering only components whose output actually differs.

Q15. ★What are keys in React, and why are they important in lists?

A: See Q7 — keys let React correctly match list items across renders so it can correctly track additions, removals, and reorders without confusing component identity or losing local state.

Q16. What is the difference between an element and a component in React?

A: A component is a function or class that defines how to render UI, while an element is the lightweight, immutable plain object (created by JSX or React.createElement) describing what to render — essentially a

description/instruction for React, not the actual rendered instance. Calling a component produces elements; React interprets those elements to create and manage component instances.

Q17. ★What is a 'controlled' vs 'uncontrolled' component?

A: See Q8 — controlled components source their value from React state and require an explicit update handler, while uncontrolled components track their own value internally, typically accessed via a ref, giving up single-source-of-truth state in exchange for simplicity.

Q18. What is the difference between React Native's StyleSheet and inline styles?

A: StyleSheet.create() defines styles once, and React Native can reference them by numeric ID, sending them across the bridge only once and allowing some validation/optimization; inline style objects are recreated on every render, which is less efficient and harder to reuse or override consistently.

2. Functional Components & JSX

Questions on writing and structuring components the modern way.

Q19. ★Why are functional components now preferred over class components?

A: Functional components with hooks are more concise, avoid the confusing `this` binding of classes, make it easier to reuse stateful logic (via custom hooks instead of HOCs/render props), and are the primary focus of React's ongoing feature development (e.g. Concurrent features, React Compiler).

Q20. What does it mean that a functional component is 'just a function that returns JSX'?

A: A functional component is a plain JavaScript function that takes props as an argument and returns a description of UI (JSX, which compiles to React.createElement calls). React calls this function on each render to determine what should be displayed.

Q21. ★How do you conditionally render UI in JSX (ternaries, &&, early returns)?

A: You can use a ternary (`condition ? <A/> : <B/>`) to choose between two outputs, the `&&` operator (`condition && <A/>`) to render something or nothing, or an early `return` inside the component to short-circuit rendering entirely based on a condition, such as returning a loading spinner before the main JSX.

Q22. ★What's the danger of using && for conditional rendering with numeric values (e.g. count && <Text>...</Text>)?

A: If `count` is `0`, `count && <Text>...</Text>` evaluates to `0` (a falsy but non-boolean value), and React will render the literal text "0" on screen instead of nothing. The fix is to convert to a boolean explicitly, e.g. `count > 0 && ...` or `Boolean(count) && ...`.

Q23. How do you render a list of items in JSX, and what role does .map() play?

A: You call `.map()` on an array to transform each item into a JSX element, producing an array of elements that JSX can render directly, e.g. `items.map(item => <Text key={item.id}>{item.name}</Text>)`. Each element in the resulting array needs a unique `key` prop.

Q24. Why can a component only return a single root element (or a Fragment)?

A: React's render output needs to be one tree, and a component returning multiple sibling elements without a common wrapper would be ambiguous to represent as a single object; JSX enforces this by requiring one root node, which can be a wrapping element or a Fragment when you don't want extra markup.

Q25. What is a React Fragment (<>...</>), and when would you use it?

A: A Fragment groups multiple children without adding an extra DOM/native node to the tree. It's used when a component needs to return multiple siblings (e.g. table rows or list items) but wrapping them in a real element like a View would break layout or semantics.

Q26. ★How do you pass children into a component, and what is props.children?

A: Anything nested between a component's opening and closing JSX tags is automatically passed to it as the special `children` prop. Inside the component you access it via `props.children` and render it wherever you want, enabling reusable wrapper/layout components.

Q27. What's the difference between a presentational (dumb) component and a container (smart) component?

A: A presentational component focuses purely on how things look, receiving data and callbacks via props with no knowledge of where they come from, while a container component handles data fetching, state, and business logic and passes the results down to presentational components. Hooks have blurred this distinction, but it remains a useful way to separate concerns.

Q28. How do default parameter values work when destructuring props in a functional component?

A: You can assign a fallback directly in the destructuring pattern, e.g. `function Button({ label = 'Submit' }) {}`, so if the caller doesn't pass that prop (or passes `undefined`), the default value is used instead.

Q29. ★Why should you avoid defining a new component type inside another component's render/body?

A: Defining a component inside another component's body creates a brand-new function/type on every parent render, so React treats it as an entirely different component type each time, unmounting and remounting the child (losing its state and possibly hurting performance) instead of reconciling it as the same component.

Q30. What are Higher-Order Components (HOCs), and how do they differ from hooks-based composition?

A: An HOC is a function that takes a component and returns a new, enhanced component, historically used to share cross-cutting logic like data fetching or theming. Hooks achieve similar reuse without wrapping components or adding extra layers to the tree, making the data flow more direct and avoiding issues like prop name collisions ('wrapper hell').

Q31. What is a render prop pattern, and is it still commonly used?

A: A render prop is a prop whose value is a function that a component calls to determine what to render, letting the consumer control rendering while the component supplies data or behavior. It's largely been superseded by custom hooks, which offer the same logic-sharing benefit with simpler syntax and no extra nesting in the tree.

Q32. How do you split a large component into smaller, reusable pieces?

A: Identify logically distinct pieces of UI or responsibility (e.g. a header, a list item, a form field) and extract each into its own component with clearly defined props; shared non-UI logic (data fetching, subscriptions) can be extracted into custom hooks so both the UI and the logic stay small and testable.

3. Props

How data flows down the component tree.

Q33. ★What are props, and how do they differ from state?

A: Props are read-only data passed into a component from its parent, used to configure how it renders. State is data owned and managed internally by a component that can change over time and trigger re-renders — props flow down and are controlled externally, while state is local and controlled internally.

Q34. ★Why are props described as 'read-only' / immutable?

A: A component must never mutate the props object it receives, because that object may be shared with the parent's own state or other consumers, and mutating it would break React's ability to detect changes and lead to unpredictable rendering. If a value needs to change, it should live in state and be passed down as a new prop.

Q35. ★How do you pass a function as a prop, and why is this pattern used to 'lift state up'?

A: You pass a function reference as a prop just like any other value, e.g. `<Child onSave={handleSave} />`, and the child calls it (often with arguments) to notify the parent of an event. This lets a child communicate upward, which is how 'lifting state up' works: the parent owns the state and passes both the value and an updater function down.

Q36. ★What is prop drilling, and what problems does it cause in a deeply nested component tree?

A: Prop drilling is passing a prop through many intermediate components that don't use it themselves, just to get it to a deeply nested descendant. It makes components harder to reuse and refactor, clutters intermediate components' APIs with irrelevant props, and increases the chance of bugs when a prop's name or shape changes.

Q37. How do you set default values for props?

A: In modern function components, defaults are set via default parameter syntax during destructuring, e.g. `function Card({ title = 'Untitled' }) {}`. (Class components historically used `defaultProps`, which is now deprecated for function components.)

Q38. How does destructuring props ({ name, age }) improve readability compared to using props.name?

A: Destructuring surfaces exactly which props a component uses right in the function signature, removing the repetitive `props.` prefix throughout the body and making it immediately clear (and easier to spot missing/renamed props) what data the component depends on.

Q39. What is props.children, and how would you build a reusable wrapper/layout component with it?

A: `props.children` holds whatever JSX was nested inside a component's tags. A wrapper component like `Card` can render `props.children` inside its own markup (e.g. inside padding and a border), letting callers supply arbitrary content while the wrapper controls the surrounding layout and styling.

Q40. ★How would you type props in TypeScript for a React Native component?

A: Define a `type` or `interface` describing the prop names and types, e.g. `interface ButtonProps { label: string; onPress: () => void; disabled?: boolean }`, then annotate the component as `function Button(props: ButtonProps)` or use it as the generic parameter for `React.FC<ButtonProps>` if you choose that style.

Q41. ★What happens if a parent re-renders — do all children automatically re-render too, and why?

A: By default, yes: when a parent component re-renders, React calls all of its child components' functions again, regardless of whether their props actually changed, because React's default behavior is to re-render the whole subtree. `React.memo` can opt a child out of this by skipping re-render when props are shallowly equal.

Q42. How do you pass multiple values (like an object) as a single prop versus spreading individual props?

A: You can bundle related values into one object prop, e.g. `<Avatar user={user} />`, or spread an object's properties as individual props with `<Avatar {...user} />`. Passing an object keeps the API compact and groups related data, while spreading individual props makes each one explicit and easier to type-check independently.

Q43. What's the difference between passing a prop by value vs by reference, and why does that matter for re-renders?

A: Primitives (numbers, strings, booleans) are compared by value, so identical values are always `===` equal across renders; objects, arrays, and functions are compared by reference, so a newly created object/array/function is never `===` equal to the previous one even with the same contents. This matters because `React.memo`, `useMemo`, and dependency arrays rely on `===` checks, so recreating reference values on every render can defeat memoization.

Q44. How would you validate prop types at runtime in a JavaScript (non-TS) React Native project?

A: You can use the `prop-types` library to declare a `propTypes` static property on a component describing expected types and required-ness; React logs a console warning in development when a component receives props that don't match, catching mismatches without needing TypeScript.

Q45. ★What are 'callback props', and how do they let a child communicate back up to a parent?

A: Callback props are function props (like `onChange` or `onPress`) that a parent passes to a child so the child can invoke them, typically with relevant data, when something happens. Since children can't directly modify a parent's state, calling a callback prop is the standard way for a child to report events upward.

Q46. Why shouldn't you mutate an object or array received as a prop?

A: The prop may be the same reference the parent is using in its own state, so mutating it directly changes the parent's data without going through `setState`, which won't trigger a re-render and can cause the UI to desync from the underlying data. Instead you should create a new object/array with the desired changes and pass that up via a callback.

4. State & useState

Component-local memory and how updates trigger re-renders.

Q47. ★What is state, and how is it different from a regular JavaScript variable inside a component?

A: State is data that React tracks and persists across renders for a component, and updating it schedules a re-render. A plain variable declared in the component body is reset to its initial value on every render and updating it doesn't cause React to re-render or reflect the change on screen.

Q48. ★How does calling a useState setter trigger a re-render?

A: Calling the setter function schedules an update: React marks the component as needing to re-render, stores the new value, and re-invokes the component function on the next render pass so it returns updated JSX reflecting the new state. React may batch multiple setter calls together before re-rendering once.

Q49. ★Why should you use the functional updater form (setCount(prev => prev + 1)) instead of setCount(count + 1)?

A: The functional form receives the most current state value at the time the update is actually applied, avoiding stale-closure bugs when multiple updates are batched or queued in the same tick — `setCount(count + 1)` uses the `count` captured in that render's closure, which can be outdated if called multiple times before a re-render happens.

Q50. ★Are state updates synchronous or asynchronous/batched in React?

A: State updates don't apply immediately to the variable in the current render's closure; React batches multiple `setState` calls (in event handlers, and since React 18 in most other contexts too, like timeouts and promises) into a single re-render for efficiency, so you shouldn't expect to read the updated value right after calling the setter.

Q51. What happens if you call a state setter with the exact same value it already holds?

A: React uses `Object.is` comparison to bail out: if the new value is referentially/value-equal to the current state, React skips re-rendering that component (though it still exits any batching early), avoiding unnecessary work.

Q52. ★How do you manage an array or object in state without directly mutating it?

A: Always create a new array/object rather than mutating the existing one — e.g. use spread syntax (`setItems([...items, newItem])`), array methods that return new arrays (`map`, `filter`), or object spread (`setUser({ ...user, name: 'New' })`) — so React can detect the reference change and re-render correctly.

Q53. What's the difference between useState(initialValue) and useState(() => computeInitialValue()) (lazy initialization)?

A: Passing a plain value runs that expression on every render (even though it's only used on the first), while passing a function defers computation so React only calls it once, on the initial render — useful when computing the initial value is expensive.

Q54. Why does state 'reset' when a component unmounts and remounts?

A: React state is stored per component instance in the tree; when a component is unmounted (removed from the tree), that instance and its associated state are discarded entirely, so if the same component type mounts again later, it starts fresh with its initial state.

Q55. ★How would you decide whether a piece of data belongs in state, or can be derived during render?

A: If a value can be computed directly from existing props/state during render (e.g. a filtered list, a total, a formatted string), it should be derived on the fly rather than stored in its own state — storing it redundantly risks it getting out of sync with the source data it was derived from.

Q56. What issues arise from storing redundant/derived state instead of computing it on the fly?

A: Redundant state can drift out of sync with its source data if you forget to update it every place the source changes, leading to subtle bugs, extra re-renders, and more code to keep values consistent than simply recomputing the derived value each render.

Q57. How does React batch multiple setState calls within the same event handler?

A: React collects all state updates triggered synchronously within the same event handler (and, since React 18, within most async callbacks too) and applies them together in a single re-render pass, rather than re-rendering after each individual call, improving performance and consistency.

Q58. What happens to state when a component's key changes in a list?

A: Changing a component's `key` tells React it's a different logical element, so React unmounts the old instance (discarding its state) and mounts a brand-new instance with fresh state, even if the component type and other props are identical — a common technique to intentionally reset a component.

Q59. ★How would you lift state up between two sibling components?

A: Move the state to their common parent component, then pass the value down to both children as props, and pass a callback prop down to whichever child needs to update it; the parent's setter updates the shared state, triggering both siblings to re-render with the new value.

Q60. What's a common bug when storing an object in state and updating only one field?

A: Mutating the object directly (e.g. `user.name = 'New'; setUser(user)`) doesn't create a new reference, so React may not detect a change and skip re-rendering; the fix is to spread the existing object and override just the changed field, e.g. `setUser({ ...user, name: 'New' })`.

5. useEffect & Side Effects

Handling data fetching, subscriptions, timers, and cleanup.

Q61. ★What is a 'side effect' in React, and why can't you just do it directly in the component body?

A: A side effect is any operation that reaches outside the pure render calculation — data fetching, subscriptions, timers, manually touching the DOM/native APIs, logging, etc. Doing these directly in the component body would run them on every single render (including ones caused just by re-rendering, not real changes) and could run during React's rendering phase in ways that produce inconsistent results, so `useEffect` runs them safely after render instead.

Q62. ★What is the dependency array, and how does its content control when the effect runs?

A: The dependency array is the second argument to `useEffect`, listing the reactive values the effect reads. React compares each dependency to its value from the previous render, and only re-runs the effect if at least one has changed; this lets an effect stay in sync with the specific values it depends on rather than running on every render.

Q63. ★What's the difference between no dependency array, an empty array [], and an array with values?

A: No array means the effect runs after every render; an empty array `[]` means the effect runs once after the initial render and never again (since there's nothing to change); an array with values means the effect runs after the initial render and again whenever any listed value changes between renders.

Q64. ★What is the cleanup function returned from useEffect, and when does it run?

A: A cleanup function is an optional function returned from the effect callback that undoes what the effect set up (e.g. removing an event listener, clearing a timer, closing a connection). React runs it right before the effect runs again (using the previous render's values) and also when the component unmounts.

Q65. ★Why is a cleanup function essential when setting up a timer, subscription, or event listener?

A: Without cleanup, each time the effect re-runs it would add another timer/listener/subscription on top of the previous ones instead of replacing them, leading to duplicated behavior, memory leaks, and callbacks referencing stale closures — cleanup ensures exactly one active instance at a time.

Q66. ★How would you fetch data inside useEffect, including handling loading and error states?

A: Declare `loading`, `data`, and `error` state; inside the effect, set `loading` to true, call the fetch (often with a cancellation guard), then on success set `data` and `loading` false, and on failure set `error` and `loading` false, all inside a try/catch or `.then/.catch` chain, with the fetch triggered by the appropriate dependency (e.g. an ID).

Q67. Why can't the function passed to useEffect itself be async?

A: `useEffect`'s callback is expected to either return nothing or return a synchronous cleanup function; an `async` function implicitly returns a Promise instead, which React would incorrectly try to treat as (or warn about) a cleanup function. The fix is to define an inner `async` function and call it inside the effect.

Q68. ★How do you prevent a state update on an unmounted component after an async fetch resolves?

A: Use a boolean flag or an `AbortController` set up in the effect and checked/cleaned up in the cleanup function — e.g. set `let cancelled = false` and return `() => { cancelled = true }`, then only call the state setter if `!cancelled` once the async operation resolves.

Q69. ★What is a race condition in data fetching, and how does a 'cancelled' flag or AbortController prevent it?

A: A race condition happens when multiple requests are in flight (e.g. triggered by rapidly changing a dependency like a search query) and an older, slower request resolves after a newer one, overwriting fresher data with stale results. A cancel flag or `AbortController` set in the effect's cleanup ensures that when a newer effect run starts, the previous request's result is ignored or the request itself is aborted.

Q70. ★What are the ESLint exhaustive-deps warnings, and why shouldn't you just silence them?

A: The `react-hooks/exhaustive-deps` rule flags when an effect uses a value that isn't listed in its dependency array, since that usually means the effect will use a stale closure value instead of the current one. Silencing the warning without fixing the underlying issue is a common source of subtle bugs where effects don't react to data they should.

Q71. How do you run an effect only once, on mount, and never again?

A: Pass an empty dependency array, `useEffect(() => { ... }, [])`; React then runs the effect after the first render only, since there are no dependencies that could change to trigger a re-run.

Q72. What's the difference between useEffect and useLayoutEffect, and when would you need the latter?

A: `useEffect` runs asynchronously after the browser has painted the screen, while `useLayoutEffect` runs synchronously after DOM mutations but before the browser paints. You'd use `useLayoutEffect` when you need to measure or mutate the DOM (e.g. read an element's size and adjust layout) before the user sees any visual flicker.

Q73. How would you debounce a search input using useEffect?

A: Set a `setTimeout` inside the effect that fires the actual search after a delay (e.g. 300ms) whenever the query state changes, and return a cleanup function that calls `clearTimeout` on that timer — so if the query changes again before the delay elapses, the pending search is cancelled and a new timer starts.

Q74. ★How do you avoid an infinite loop caused by updating state inside an effect that depends on that same state?

A: Use the functional updater form (`setX(prev => ...)`) so you don't need to list `x` itself as a dependency, or restructure the logic so the effect depends on a different, more stable value, or move the logic to run in response to a specific event instead of as a reactive effect on that state.

Q75. How do you fetch data again when a prop like userId changes?

A: Include `userId` in the effect's dependency array; whenever a re-render occurs with a different `userId` value, React re-runs the effect (after cleaning up the previous run), triggering a fresh fetch for the new ID.

Q76. ★What's the danger of putting an object or array literal directly in the dependency array?

A: An inline object/array literal (e.g. `{ id }` or `[a, b]`) is a new reference on every render even if its contents are identical, so React's shallow `Object.is` comparison sees it as 'changed' every time, causing the effect to re-run on every render — the fix is to depend on the primitive values inside it, or memoize the object with `useMemo`.

Q77. How would you set up and tear down a WebSocket connection using useEffect?

A: Inside the effect, create the WebSocket and attach message/error handlers; return a cleanup function that calls `socket.close()` so the connection is properly torn down whenever the effect re-runs (e.g. if the URL dependency changes) or the component unmounts, preventing leaked open connections.

Q78. Can a component have multiple useEffect calls, and why might you split effects instead of combining them?

A: Yes — a component can use `useEffect` as many times as needed. Splitting effects by concern (e.g. one for a subscription, another for logging) keeps each effect's dependency array focused and correct, makes cleanup logic clearer, and avoids one effect re-running unnecessarily because of a dependency it doesn't conceptually care about.

6. useReducer & Complex State

Managing state with multiple sub-values or complex transition logic.

Q79. ★What is useReducer, and when would you reach for it instead of useState?

A: `useReducer` manages state via a reducer function that computes the next state from the current state and a dispatched action, similar to Redux. It's preferable to `useState` when state is complex (multiple related sub-values), when the next state depends on detailed logic around the previous state, or when many different event types update the same state in structured ways.

Q80. ★What are the three core parts of the reducer pattern: state, action, and dispatch?

A: State is the current data snapshot; an action is a plain object (typically with a `type` and optional `payload`) describing what happened; `dispatch` is the function you call with an action to trigger the reducer, which computes and returns the new state that React then uses to re-render.

Q81. ★Why must a reducer function be 'pure' (no side effects, no mutation)?

A: React may call reducers multiple times (e.g. in Strict Mode, or to replay them for concurrent features) and relies on them being deterministic — same state and action always produce the same new state — with no side effects and no mutation of the existing state object, so that React's internal bookkeeping and time-travel/debug tooling stay correct.

Q82. How do you structure an action object, and what is the role of a type field?

A: An action is typically `{ type: 'ADD_ITEM', payload: item }`; `type` is a string identifying which kind of transition should occur, and the reducer switches on it to decide how to compute the new state, while `payload` (or other named fields) carries any additional data the transition needs.

Q83. How would you handle an unknown action type in a switch statement inside a reducer?

A: Add a `default` case that either returns the current state unchanged (a safe no-op) or throws an error to surface bugs during development — throwing is often preferred in reducers so an unrecognized action type is caught immediately rather than silently ignored.

Q84. How do you pass additional data to a reducer via an action's payload?

A: Include the extra data as a `payload` property (or more specific named fields) on the action object when dispatching, e.g. `dispatch({ type: 'UPDATE_QTY', payload: { id, qty: 3 } })`, and the reducer reads `action.payload` to compute the new state.

Q85. ★How would you model a shopping cart's add/remove/update-quantity logic using useReducer?

A: Define actions like `ADD_ITEM`, `REMOVE_ITEM`, and `UPDATE_QUANTITY`, each carrying the relevant item id/quantity as payload; the reducer switches on `action.type` and returns a new cart array — appending a new item, filtering out a removed one, or mapping over items to update the matching one's quantity — without mutating the previous array.

Q86. ★What's the advantage of useReducer for state where 'the next state depends on the previous state' in complex ways?

A: Centralizing all transition logic in one reducer function makes complex state changes explicit, testable in isolation, and consistent — instead of scattering multiple `setState` calls with intertwined logic across event handlers, every possible transition is defined in one place and dispatched via simple action objects.

Q87. ★How does useReducer compare to Redux, and when would you graduate from one to the other?

A: `useReducer` follows the same reducer/action/dispatch pattern as Redux but is scoped to a single component (or shared via Context), with no middleware, devtools, or cross-component store by default. You'd graduate to Redux (or a similar library) when you need state shared broadly across the app, middleware for async logic, time-travel debugging, or strong conventions for a large team.

Q88. How would you calculate a derived total (like totalItems) inside a reducer instead of recomputing it separately?

A: You can store the derived value directly in the reducer's state and recompute it as part of every relevant action (e.g. recalculate `totalItems` whenever items are added/removed within the same reducer transition), keeping it always consistent with the rest of the state in a single atomic update.

Q89. ★Can you combine useReducer with useContext to build a lightweight global store? How?

A: Yes — create a Context, instantiate `useReducer` once in a top-level provider component, and pass both `state` and `dispatch` down through the Context's value. Any descendant component can then read state and dispatch actions via `useContext`, giving you a Redux-like global store without an external library.

Q90. How do you reset state back to its initial value using a reducer action?

A: Add a `RESET` action type whose case in the reducer returns the original `initialState` object (kept as a constant outside the component so it's stable), e.g. `case 'RESET': return initialState;`, then dispatch `{ type: 'RESET' }` whenever you want to restore defaults.

Q91. What are the trade-offs of putting business logic inside the reducer vs. inside event handlers?

A: Putting logic in the reducer centralizes and makes state transitions pure, testable, and consistent regardless of where dispatch is called from, but reducers can't easily perform side effects like API calls; putting logic in event handlers keeps reducers simple but risks duplicating transition logic across multiple handlers and mixing side effects with state updates.

Q92. How would you test a reducer function in isolation?

A: Since a reducer is a pure function, you can call it directly in a unit test with a given state and action and assert on the returned state, without needing to render any component — e.g. `expect(reducer(initialState, { type: 'ADD_ITEM', payload })).toEqual(expectedState)`.

7. Context API & useContext

Sharing data across the component tree without prop drilling.

Q93. ★What problem does the Context API solve, and how does it relate to prop drilling?

A: Context lets you share a value (like a theme, logged-in user, or locale) with any component in a subtree without manually passing it down through every intermediate layer as a prop. It directly addresses prop drilling by letting deeply nested components subscribe to the value directly.

Q94. ★What are the three main pieces involved in using Context: createContext, Provider, and useContext?

A: `createContext(defaultValue)` creates a Context object; the `Provider` component (`<MyContext.Provider value={...}>`) wraps part of the tree and supplies the actual value to all descendants; `useContext(MyContext)` is called inside any descendant component to read the current value provided by the nearest matching Provider above it.

Q95. What is the default value passed to createContext(), and when does it actually get used?

A: The default value is used only when a component calls `useContext` without any matching Provider above it in the tree; if a Provider is present, its `value` prop always takes precedence regardless of the default passed to `createContext`.

Q96. ★Why is it a common pattern to wrap useContext in a custom hook (e.g. useTheme())?

A: A custom hook like `useTheme()` hides the Context object as an implementation detail, gives consumers a cleaner and more descriptive API, and is a convenient place to add a runtime check that throws a helpful error if the hook is used outside its Provider.

Q97. ★What happens to all consuming components when a Context Provider's value changes?

A: Every component that calls `useContext` on that Context re-renders when the Provider's `value` changes (based on reference equality), regardless of whether that particular component actually uses the part of the value that changed — Context does not do fine-grained subscription by default.

Q98. ★Why should the value passed to a Provider be memoized (e.g. with useMemo)?

A: If the `value` prop is a new object literal created on every render of the Provider's parent, every consumer sees a 'changed' value and re-renders even when the underlying data hasn't actually changed; wrapping it in `useMemo` (with correct dependencies) keeps the reference stable across renders where nothing relevant changed.

Q99. ★What's the performance downside of putting too much unrelated state into a single Context?

A: Any change to any piece of a large combined Context value re-renders every consumer of that Context, even ones that only care about an unrelated slice of it, which can cause widespread unnecessary re-renders across the app as the Context grows to hold more unrelated data.

Q100. How would you split Context into multiple smaller providers to avoid unnecessary re-renders?

A: Break state into logically separate Contexts by concern (e.g. `ThemeContext`, `UserContext`, `CartContext`) so a component only subscribes to — and only re-renders due to changes in — the specific Context it actually needs, rather than one large combined Context.

Q101. How do you nest multiple Context Providers (e.g. Theme + User) around an app?

A: Wrap providers around each other at the root of the tree, e.g. `<ThemeProvider><UserProvider><App /></UserProvider></ThemeProvider>`, or build a small composite `AppProviders` component that nests them internally so `App` itself stays clean; order generally doesn't matter unless one provider depends on another's context.

Q102. ★Is Context a replacement for a state management library like Redux? Why or why not?

A: Context is a dependency-injection mechanism for passing a value down the tree — it doesn't provide middleware, selectors/fine-grained subscriptions, devtools, or optimized update batching the way libraries like Redux or Zustand do. It works well combined with `useReducer` for small-to-medium global state, but larger apps with complex or frequently-updating shared state often still benefit from a dedicated state library.

Q103. How would a deeply nested component update the value held in Context (not just read it)?

A: The Provider's value should include not just the data but also an updater function (e.g. a `setUser` or `dispatch` from `useReducer`/`useState` defined in the provider component), so any consumer can call that function via `useContext` to trigger an update at the source.

Q104. What error-handling pattern would you add to a custom useX() hook to catch 'used outside Provider' bugs?

A: Have `createContext` default to `undefined` (or a sentinel), then in the custom hook check if the returned context is that sentinel and `throw new Error('useX must be used within an XProvider')` if so, turning a silent bug (using default/undefined data) into an immediate, clear error during development.

Q105. ★How does Context interact with React.memo — can a memoized child still re-render due to Context changes?

A: Yes — `React.memo` only prevents re-renders caused by unchanged props from a parent; it has no effect on re-renders triggered by a Context value changing, since Context reads happen inside the component itself via `useContext`, bypassing the props-based memoization check entirely.

Q106. When would prop drilling actually be preferable to Context?

A: For shallow trees or when only one or two intermediate components pass a value down, explicit props keep data flow easy to trace and don't introduce Context's implicit re-render behavior or extra indirection — Context pays off more clearly as nesting depth and the number of consumers grow.

8. Performance: memo, useCallback & useMemo

Avoiding unnecessary re-renders, especially in long lists.

Q107. ★What does React.memo do, and how does it decide whether to skip a re-render?

A: `React.memo` wraps a component so React skips re-rendering it when its parent re-renders, as long as its props are shallowly equal (each prop compared with `===`) to the previous render's props; if all props are unchanged by reference, React reuses the previous render output instead of calling the component again.

Q108. ★Why does wrapping a component in memo alone often fail to prevent re-renders?

A: If the parent passes new object, array, or function props on every render (e.g. an inline arrow function or object literal), those props are never `===` equal to the previous ones even though their contents look the same, so `memo`'s shallow comparison sees them as changed and re-renders anyway — `memo` needs stable prop references to be effective.

Q109. ★What does useCallback do, and what problem does it solve for memoized children?

A: `useCallback` memoizes a function definition itself, returning the same function reference across renders as long as its dependency array values haven't changed. This keeps a callback prop's reference stable, which allows a `React.memo`-wrapped child receiving that callback to actually skip re-rendering instead of seeing a 'new' function every time.

Q110. ★Why does an inline arrow function passed as a prop break React.memo?

A: An inline arrow function (e.g. `onPress={() => doThing()}`) is a brand-new function object created on every render, so it's never reference-equal to the previous render's function even if it behaves identically — this defeats `React.memo`'s shallow prop comparison on the child receiving it.

Q111. ★What's the difference between useCallback and useMemo?

A: `useCallback(fn, deps)` memoizes and returns the function itself, while `useMemo(() => value, deps)` memoizes and returns the result of calling a function (a computed value). `useCallback(fn, deps)` is essentially equivalent to `useMemo(() => fn, deps)`.

Q112. ★When would you use useMemo to avoid recomputing an expensive derived value?

A: Use it when a calculation is genuinely costly (e.g. filtering/sorting a large array, complex aggregation) and its inputs don't change on every render, so you want to skip re-running that computation unless one of its listed dependencies actually changes.

Q113. ★Why is it important to memoize the renderItem function passed to a FlatList?

A: `FlatList` re-renders list items when `renderItem` changes reference, so passing a new inline function on every render of the parent causes all visible rows to re-render even if their own data hasn't changed; wrapping `renderItem` in `useCallback` keeps it stable and lets rows skip unnecessary re-renders.

Q114. What is React.memo's default comparison behavior, and how do you customize it with a second argument?

A: By default `React.memo` does a shallow equality check on each prop. You can pass a custom comparison function as the second argument, `React.memo(Component, (prevProps, nextProps) => boolean)`, returning `true` if the props are 'equal enough' to skip re-rendering, giving finer control than the default shallow check.

Q115. ★Why can premature or excessive memoization actually hurt performance?

A: `useMemo`/`useCallback` themselves have a small cost (storing and comparing dependencies each render), and `React.memo`'s comparison also costs something; applying them everywhere — including on cheap components or trivial computations — can add overhead and code complexity without any measurable rendering benefit, sometimes making things slightly slower.

Q116. ★How would you use the functional updater form of useState to remove a dependency and keep a useCallback stable forever?

A: Instead of writing `const inc = useCallback(() => setCount(count + 1), [count])` (which changes every time `count` changes), write `const inc = useCallback(() => setCount(prev => prev + 1), [])`, since the updater function receives the latest state itself, letting the callback have an empty dependency array and a permanently stable reference.

Q117. ★How do you measure whether a component is actually re-rendering unnecessarily (e.g. console logs, React DevTools Profiler)?

A: You can add a `console.log` or a render counter inside the component body to see how often it runs, or use the React DevTools Profiler tab to record a session and inspect which components rendered, how long each took, and why (it can show the reason, such as 'props changed' or 'state changed'), helping pinpoint avoidable re-renders.

Q118. What's the difference between a component re-rendering and it actually re-painting/updating the DOM/native view?

A: Re-rendering means React calls the component function again and produces a new element tree; this doesn't necessarily change the actual DOM/native view — if the reconciled output is identical to before, React skips the underlying mutation. So a component can 're-render' in React's terms without any visible repaint or native update actually occurring.

Q119. ★How does object/array identity (===) affect dependency arrays and memoized props?

A: React's dependency-array comparisons and `React.memo`'s default prop comparison both use `===`, which for objects/arrays checks reference identity, not deep content equality; a newly created object/array with identical contents still fails this check, which is why unmemoized objects/arrays passed as dependencies or props routinely cause unnecessary re-runs or re-renders.

Q120. What is the extraData prop on FlatList, and why is it needed even with a memoized renderItem?

A: `FlatList` internally memoizes rows and by default only re-renders a row when the row's own data or its stable `renderItem` reference changes; if a row needs to reflect some external state that isn't part of its own item data (e.g. a globally selected item id), passing that value via `extraData` tells `FlatList` to re-render rows when it changes too.

Q121. How would you profile and fix a slow-scrolling FlatList with many complex rows?

A: Use the React DevTools Profiler or in-app performance monitor to find what's slow, then apply fixes such as memoizing `renderItem`/row components, simplifying row markup, using `getItemLayout` to skip dynamic measurement, tuning `windowSize`/`initialNumToRender`, avoiding expensive computations or images at full resolution per row, or switching to a more optimized list like `FlashList`.

Q122. What's the difference between memoizing a value and memoizing a component?

A: Memoizing a value (`useMemo`) caches the result of a computation so it isn't recalculated unnecessarily; memoizing a component (`React.memo`) caches the component's rendered output so React can skip re-executing and re-reconciling that component's function when its props haven't meaningfully changed.

Q123. Can you over-rely on useCallback/useMemo in a way that makes code harder to read for little benefit?

A: Yes — wrapping every function and value in `useCallback`/`useMemo` 'just in case' adds visual noise, dependency-array upkeep, and cognitive overhead throughout a codebase, often with no measurable performance gain for components that render cheaply or infrequently; it's best reserved for cases with a demonstrated performance need.

Q124. How does React 19's compiler ('React Compiler') aim to reduce the need for manual memoization?

A: The React Compiler analyzes your component code at build time and automatically inserts the equivalent of `useMemo`/`useCallback`/`React.memo` optimizations where safe, based on static analysis of what values and functions actually change, so developers can write plain code and let the compiler handle fine-grained memoization instead of doing it by hand.

9. Lists & FlatList

Rendering large, scrollable, dynamic collections efficiently.

Q125. ★Why does React Native discourage using .map() inside a ScrollView for long lists?

A: A `ScrollView` renders all of its children immediately, regardless of whether they're visible on screen, so mapping a large array inside one creates and keeps every row mounted in memory and in the native view hierarchy at once, which hurts memory usage, initial render time, and scroll performance as the list grows.

Q126. ★How does FlatList achieve better performance than a plain ScrollView with mapped items?

A: `FlatList` virtualizes its content: it only renders items currently visible (plus a small buffer/'window') on screen, recycling/unmounting rows that scroll far out of view and mounting new ones as they scroll into view, so memory and rendering cost stay roughly constant regardless of total list length.

Q127. ★What is the purpose of the keyExtractor prop, and what happens if keys aren't stable/unique?

A: `keyExtractor` tells `FlatList` how to derive a unique, stable key for each item (by default it looks for a `key` or `id` field). If keys aren't unique or stable across renders, `FlatList` can misidentify items during re-renders/reordering, causing incorrect row reuse, lost local state, or subtle rendering bugs.

Q128. What does 'windowing' or 'virtualization' mean in the context of list rendering?

A: Windowing/virtualization means only rendering a subset of a large dataset — the items currently near the viewport — instead of the entire list at once, dynamically mounting and unmounting rows as the user scrolls so the amount of rendered content stays bounded regardless of the total data size.

Q129. What are initialNumToRender, windowSize, and maxToRenderPerBatch, and how do they affect performance?

A: `initialNumToRender` controls how many items render on first mount (a smaller number speeds up initial load); `windowSize` controls how many 'screens' worth of content are kept rendered above/below the viewport (a larger window means smoother fast-scrolling but more memory use); `maxToRenderPerBatch` controls how many items are rendered per incremental batch as the user scrolls, trading off batch render smoothness against how quickly new content appears.

Q130. How would you implement pull-to-refresh and infinite scrolling with FlatList?

A: For pull-to-refresh, pass `refreshing` (boolean state) and `onRefresh` (a callback that re-fetches data and resets the flag) to `FlatList`. For infinite scrolling, pass `onEndReached` (fires when the user nears the list's end) and `onEndReachedThreshold` to trigger fetching and appending the next page of data to the list's state.

Q131. What's the difference between FlatList, SectionList, and FlashList?

A: `FlatList` renders a single flat virtualized array; `SectionList` extends the same virtualization to grouped data with section headers (like a contacts list grouped by letter); `FlashList` is a community/Shopify-built drop-in replacement for `FlatList` that uses cell recycling for significantly better scroll performance, especially with large or complex lists.

Q132. How do you render an empty state when a FlatList's data array is empty?

A: Pass a `ListEmptyComponent` prop with the UI (e.g. an illustration and message) you want shown when `data` is an empty array; `FlatList` automatically displays it instead of any rows in that case.

Q133. ★Why should renderItem avoid creating new inline functions/objects on every render?

A: If `renderItem` creates new inline callbacks or style objects on every invocation, any memoized row component receiving them can't bail out of re-rendering (their props are never `===` equal), undermining the performance benefit of virtualization and memoization — stable references (via `useCallback`/`useMemo`/`StyleSheet.create`) let rows actually skip unnecessary work.

Q134. How would you add item separators or headers/footers to a FlatList?

A: Use the `ItemSeparatorComponent` prop for lines/spacing between items, and `ListHeaderComponent`/`ListFooterComponent` for content that appears once above or below the whole list (e.g. a title or a loading indicator for pagination).

Q135. What's getItemLayout, and when would you use it to speed up scrolling?

A: `getItemLayout` is a function you provide that tells `FlatList` the exact height/offset of each item upfront, letting it skip dynamic measurement and jump directly to any scroll position (e.g. via `scrollToIndex`). It's most useful when all rows have a fixed, known height, giving a meaningful performance and correctness boost for jump-to-item scrolling.

Q136. ★How do you filter or sort the data displayed by a list without mutating the original state array?

A: Derive a new array with `array.filter(...)` or `[...array].sort(...)` (since `.sort()` mutates in place, spread first) rather than modifying the original state array directly, and either compute this derived array during render (optionally with `useMemo`) or store the filter/sort criteria in state and recompute the display list from the source data.

Q137. How would you implement swipe-to-delete on a list item?

A: Wrap each row in a swipeable gesture component (e.g. `react-native-gesture-handler`'s `Swipeable`, or a custom `PanResponder`/reanimated gesture) that reveals a delete action or triggers a callback past a swipe threshold; on confirmation, remove that item's id from the underlying state array (immutably) so the list re-renders without it.

Q138. ★Why might using an item's array index as its key cause bugs when the list is filtered or reordered?

A: An index-based key is tied to position, not identity, so when items are reordered, inserted, or removed, the same index now refers to a different logical item; React then incorrectly reuses component instances (and their local state, like a text input's cursor or a checkbox's animation) across what are actually different pieces of data.

10. TypeScript with React Native

Typing components, props, hooks, and API responses.

Q139. ★Why is TypeScript commonly used in React Native projects, and what problems does it catch early?

A: TypeScript adds static type checking that catches errors like passing the wrong prop type, misspelling an object field, or forgetting a required argument at compile/edit time rather than at runtime on a device. It also improves editor autocomplete and makes refactoring safer in larger codebases with many shared components.

Q140. ★How do you define a type or interface for a component's props?

A: Declare a `type Props = { title: string; onPress?: () => void }` or `interface Props { title: string; onPress?: () => void }`, then use it to type the function's parameter: `function Header(props: Props) { ... }` or destructure it as `function Header({ title, onPress }: Props) { ... }`.

Q141. What's the difference between type and interface in TypeScript, and does it matter much in practice?

A: `interface` supports declaration merging (multiple declarations with the same name combine) and is generally preferred for object shapes meant to be extended, while `type` can represent unions, intersections, primitives, and mapped types that `interface` cannot. For typical component props, either works fine, and the choice is largely a team style preference.

Q142. ★How would you type useState when the initial value doesn't fully describe the shape (e.g. useState<User[]>([]))?

A: Explicitly pass the type as a generic argument, e.g. `useState<User[]>([])` or `useState<User | null>(null)`, since TypeScript can't infer a meaningful type from an empty array or `null` alone; this ensures the setter and the state variable are correctly typed for their eventual real values.

Q143. How do you type a function passed as a prop (a callback), including its parameters and return type?

A: Describe it as a function type in the props definition, e.g. `onSave: (item: Item) => void` or `onChangeText: (text: string) => void`, specifying each parameter's type and the return type (often `void` for event-style callbacks).

Q144. What is React.FC, and why have many teams moved away from using it?

A: `React.FC<Props>` is a generic type for typing a function component, which implicitly adds a `children` prop and typed static properties. Many teams avoid it because it makes `children` optional even when a component doesn't accept it, can complicate generics, and offers little benefit over simply typing the props parameter directly.

Q145. How would you type the response of a fetch() call to an API?

A: Define an interface/type describing the expected JSON shape, then assert or validate the parsed response against it, e.g. `const res = await fetch(url); const data = (await res.json()) as UserResponse;` — ideally paired with runtime validation (like Zod) since a type assertion alone doesn't guarantee the actual data matches at runtime.

Q146. ★What's the difference between a union type and an optional property when modeling a prop that might be null?

A: An optional property (`age?: number`) means the key can be entirely absent (`undefined`) but doesn't allow explicit `null` unless you add it; a union type (`age: number | null`) requires the property to be present but allows its value to explicitly be `null`, which is useful for distinguishing 'not yet loaded' (`null`) from 'not provided' (`undefined`).

Q147. How do you type children for a component, and what type does React.ReactNode represent?

A: You type it as `children: React.ReactNode` in the props type/interface. `ReactNode` is a broad type representing anything React can render — elements, strings, numbers, fragments, arrays of these, `null`, `undefined`, and booleans — making it the standard choice for a flexible children prop.

Q148. How would you type a useReducer reducer function, including the Action union type?

A: Define a `State` type, and an `Action` type as a discriminated union of each possible action shape (e.g. `{ type: 'ADD'; payload: Item } | { type: 'REMOVE'; payload: string }`), then type the reducer as `(state: State, action: Action) => State`; TypeScript will then narrow `action.payload`'s type correctly inside each `case` based on `action.type`.

Q149. ★How do you type a Context value and its default, especially when functions like setUser are involved?

A: Define an interface for the full context shape including both data and setter functions, e.g. `interface AuthContextValue { user: User | null; setUser: (u: User | null) => void }`, then create the context with `createContext<AuthContextValue | undefined>(undefined)` and narrow it (throwing if `undefined`) inside a custom `useAuth()` hook.

Q150. What are generics, and how might you use them to write a reusable typed hook or component?

A: Generics let you write functions/components/types parameterized by a type that's specified (or inferred) at the call site, preserving type safety across different concrete types. For example, a generic `useFetch<T>(url: string): { data: T | null }` hook can be reused for any endpoint while still returning strongly-typed data specific to each call.

Q151. How do you avoid using any when working with third-party or loosely-typed APIs?

A: Define your own interfaces describing the shape of data you expect and cast/validate incoming data against them (ideally with a runtime schema validator like Zod or io-ts rather than a bare type assertion), or use `unknown` combined with type guards/narrowing instead of `any` so you're forced to check the shape before using the value.

Q152. How would you type navigation params in React Navigation for type-safe screen transitions?

A: Define a `ParamList` type mapping each route name to its expected params object (or `undefined` if none), e.g. `type RootStackParamList = { Home: undefined; Profile: { userId: string } }`, then use it with the navigator's generic types and with hooks like `useNavigation<NativeStackNavigationProp<RootStackParamList>>()` so `navigate('Profile', { userId })` is checked at compile time.

11. Styling & Layout

StyleSheet, Flexbox, and platform-specific styling in React Native.

Q153. ★How does React Native's layout system (Flexbox) differ from Flexbox on the web (e.g. default flexDirection)?

A: React Native's default `flexDirection` is `column` (children stack vertically), whereas CSS Flexbox on the web defaults to `row`. React Native also implements a subset of the full CSS Flexbox spec (via Yoga) and doesn't support all web-only layout features like CSS Grid, floats, or certain flex shorthand behaviors.

Q154. What are the benefits of StyleSheet.create() over plain JavaScript style objects?

A: `StyleSheet.create()` lets React Native validate styles during development, can improve performance by referencing styles via an ID rather than serializing full objects repeatedly, and encourages defining styles once outside the render function rather than recreating style objects on every render.

Q155. How would you share common styles between light and dark themes without duplicating every property?

A: Define a base style object with properties common to both themes, then merge it with a theme-specific style object (e.g. `[styles.base, isDark && styles.dark]` or spreading `{ ...base, ...themeColors }`), so only the properties that actually differ between themes need to be declared per theme.

Q156. ★What happens in JavaScript when an object literal defines the same key twice, and how could that create a subtle styling bug?

A: JavaScript silently keeps only the last value for a duplicated key and discards the earlier one, with no error or warning; in a style object this means an earlier property (e.g. `color: 'red'` followed later by another `color: 'blue'`) is silently overridden, which can be a confusing source of 'my style isn't applying' bugs.

Q157. ★How do you conditionally apply styles based on a prop or state value (e.g. todo.completed)?

A: Pass an array of style objects to the `style` prop and include a conditional entry, e.g. `style={[styles.text, todo.completed && styles.completedText]}`; React Native merges array entries in order, so falsy values (like `false`) are simply skipped.

Q158. ★How do you combine multiple style objects, and what happens if two of them define the same property?

A: Pass an array to the `style` prop, e.g. `style={[styles.base, styles.overrides]}`; React Native merges them left to right, so if both define the same property, the value from the later object in the array wins, similar to CSS cascade order.

Q159. What units does React Native use for sizing, and how does that differ from CSS pixels?

A: React Native uses density-independent points (similar to `dp`/`pt`), unitless numbers that the platform automatically scales based on the device's pixel density, rather than raw CSS pixels; this keeps sizes visually consistent across devices with different screen densities without manual conversion.

Q160. How would you handle responsive layouts across different screen sizes and orientations?

A: Use Flexbox's relative sizing (percentages, `flex`) rather than fixed pixel dimensions where possible, read screen dimensions with the `useWindowDimensions` hook (which updates on rotation) to adapt layout decisions, and use `Platform`/breakpoint logic to render different layouts for phones vs. tablets.

Q161. What is the Platform module used for, and how would you apply a style only on iOS or only on Android?

A: `Platform.OS` (`'ios'` or `'android'`) lets you branch logic per platform, and `Platform.select({ ios: {...}, android: {...} })` lets you pick platform-specific values inline, e.g. inside a style object: `paddingTop: Platform.select({ ios: 20, android: 0 })`.

Q162. How does SafeAreaView (or safe area insets) help with notches and system UI overlap?

A: `SafeAreaView` (or the `react-native-safe-area-context` library's insets) automatically pads content away from device-specific unsafe regions like the iPhone notch/Dynamic Island, home indicator, or Android status/navigation bars, so UI elements aren't visually obscured by hardware or system chrome.

Q163. ★How would you implement a light/dark theme toggle that affects styles throughout the app?

A: Store the current theme in state or a Context/global store, derive a color palette object from it, provide that palette via a `ThemeContext` (or read it with `useColorScheme` for system-driven theming), and have components consume the palette via a `useTheme()` hook to style themselves dynamically rather than hardcoding colors.

Q164. What's the difference between padding, margin, and gap in a Flexbox layout?

A: `padding` adds space inside an element, between its border and its content/children; `margin` adds space outside an element, between it and its siblings/parent boundary; `gap` (supported in newer React Native versions) adds consistent spacing between a flex container's direct children without needing individual margins on each child.

Q165. How would you debug a layout issue where a child isn't sizing or positioning the way you expect?

A: Temporarily add background colors or borders to visualize each element's actual bounds, inspect the tree with React DevTools/the in-app layout inspector, check `flex`, `flexDirection`, `alignItems`/`justifyContent` values on parents (since a child's layout is heavily determined by its parent's flex settings), and verify there's no unexpected fixed width/height overriding flexible sizing.

Q166. Why is inline styling generally discouraged for performance and readability compared to StyleSheet.create?

A: Inline style objects are recreated on every render (a new object reference each time, which can defeat memoization) and mix presentation directly into JSX, making components harder to scan; `StyleSheet.create` defines styles once, keeps JSX cleaner, and allows React Native's tooling to optimize how styles are referenced.

12. React Native Core Components & Platform APIs

Building blocks like View, Text, TouchableOpacity, and native platform behavior.

Q167. What's the difference between View and Text, and why can't raw strings be placed inside a View directly?

A: `View` is a general-purpose container for layout (like a div) and has no built-in concept of text rendering, while `Text` is specifically responsible for displaying and styling strings and handles platform text rendering/line-wrapping. Raw strings can't go directly inside a `View` because the native layer needs to know explicitly that content is text (with font, size, etc.) rather than an arbitrary layout node, so React Native throws an error if you try.

Q168. What's the difference between TouchableOpacity, TouchableHighlight, Pressable, and when would you choose each?

A: `TouchableOpacity` dims its child's opacity on press; `TouchableHighlight` overlays an underlay color on press (common for list rows); `Pressable` is the newer, more flexible API that exposes press state so you can fully customize the pressed appearance and supports finer-grained gesture configuration. `Pressable` is generally the recommended default for new code.

Q169. How would you handle keyboard avoidance so inputs aren't hidden behind the keyboard?

A: Wrap the relevant screen content in `KeyboardAvoidingView` with an appropriate `behavior` (`padding` on iOS, `height` on Android is common), and often combine it with a `ScrollView` so the user can also scroll to reveal an input if avoidance alone isn't enough; libraries like `react-native-keyboard-aware-scroll-view` handle more edge cases automatically.

Q170. ★How does TextInput's controlled value/onChangeText pattern work, similar to a controlled form input on the web?

A: You store the input's text in state, pass it to `TextInput`'s `value` prop, and update that state in the `onChangeText` callback (which receives the new string directly, unlike web's event object); this keeps the displayed text always in sync with React state, the same controlled-component pattern used for web form inputs.

Q171. How would you show a loading spinner (ActivityIndicator) while data is being fetched?

A: Track a `loading` boolean in state, set it to `true` before starting the fetch and `false` once it resolves/rejects, and conditionally render `<ActivityIndicator />` instead of (or alongside) the main content while `loading` is true.

Q172. How do you navigate between screens using a library like React Navigation?

A: Wrap the app in a `NavigationContainer`, define a navigator (e.g. `createNativeStackNavigator`) with `Screen` entries mapping route names to components, and from within a screen call `navigation.navigate('ScreenName', params)` (obtained via the `navigation` prop or the `useNavigation` hook) to transition to another screen.

Q173. What's the difference between a stack navigator, tab navigator, and drawer navigator?

A: A stack navigator pushes/pops screens on top of each other like a call stack, typical for drill-down flows with a back button; a tab navigator shows persistent tabs (usually at the bottom) letting users switch between top-level sections; a drawer navigator shows a slide-out side menu, often used for less-frequently-accessed navigation options.

Q174. How would you persist data locally on the device (e.g. AsyncStorage or a database)?

A: For simple key-value data, use `AsyncStorage` (or the community `@react-native-async-storage/async-storage`) with its async `getItem`/`setItem` API; for more structured or larger datasets, use an embedded database like SQLite (via `expo-sqlite` or `react-native-sqlite-storage`) or a library like WatermelonDB/Realm.

Q175. How do you request and handle runtime permissions (camera, location) in React Native?

A: Use the relevant library's permission API (e.g. `expo-camera`'s `requestCameraPermissionsAsync`, or `react-native-permissions`) to prompt the user, check the returned status (granted/denied/blocked), and branch your UI accordingly — showing the feature if granted, or an explanation/settings link if denied — while also declaring the permission in the native manifest/Info.plist.

Q176. What is a native module, and when would you need to write one instead of using pure JS/React Native APIs?

A: A native module is code written in platform-native languages (Swift/Objective-C for iOS, Kotlin/Java for Android) exposed to JavaScript, used when you need functionality not available through existing JS/React Native APIs or community libraries — such as deep integration with a platform SDK, specialized hardware access, or performance-critical native code.

Q177. How would you debug a React Native app — what tools are available (Flipper, React DevTools, remote debugging)?

A: React DevTools inspects the component tree and props/state; Flipper (or its successor tooling) provides native logs, network inspection, and layout inspection; the in-app dev menu offers remote JS debugging in Chrome/Safari and performance monitors; `console.log` output appears in the Metro terminal; and native crash logs come from Xcode/Android Studio.

Q178. How does hot reloading / Fast Refresh work, and how does it differ from a full reload?

A: Fast Refresh injects updated component code into the running app while preserving component state where possible, so edits to a component's render logic appear almost instantly without losing your place in the app; a full reload restarts the entire JS bundle from scratch, resetting all state, which is needed for changes Fast Refresh can't safely apply (like editing non-component top-level code).

Q179. How would you optimize a React Native app's startup time?

A: Reduce the JS bundle size (code splitting, removing unused dependencies), enable Hermes (a lighter-weight JS engine with faster startup and precompiled bytecode), lazy-load screens/features not needed immediately, minimize synchronous work done before the first render, and use the New Architecture's TurboModules for lazy native module loading.

Q180. What's the difference between building a release (production) build and a debug build?

A: A debug build includes development-only tooling (hot reloading, the dev menu, extra warnings, unminified code) and typically loads JS from the Metro server, making it slower but easier to iterate on; a release build bundles and minifies JS ahead of time, strips development warnings/tools, and is optimized for performance and smaller size, matching what end users actually receive.

13. Architecture, Debugging & Best Practices

Higher-level judgment questions often asked in senior interviews.

Q181. How would you structure folders/files in a mid-to-large React Native project?

A: A common approach groups by feature/domain (e.g. `features/auth`, `features/profile`, each containing its own components, hooks, and screens) rather than purely by file type, alongside shared top-level folders for reusable `components`, `hooks`, `services`/`api`, `navigation`, `theme`/`styles`, and `utils`, keeping related code close together and reducing cross-folder churn as the app grows.

Q182. ★How do you decide between local component state, Context, and a global state library (Redux/Zustand/Jotai)?

A: Use local state when data is only needed by one component or its immediate children; use Context when a value needs to reach many components across a subtree but updates infrequently and doesn't need advanced tooling; reach for a global state library when state is large, updates frequently, is needed broadly across the app, or benefits from middleware, selectors, and devtools that Context alone doesn't provide.

Q183. ★What's your approach to writing custom hooks to share logic between components?

A: Identify a self-contained piece of stateful logic used in more than one place (e.g. data fetching, form handling, a subscription), extract it into a function prefixed with `use` that internally calls other hooks, and have it return only the values/functions consumers need — keeping the hook focused on one responsibility and independent of any specific UI.

Q184. How would you test a React Native component (unit tests, snapshot tests, integration tests)?

A: Use Jest with React Native Testing Library to render components and assert on behavior from the user's perspective (querying by text/role and simulating presses/input) rather than implementation details; snapshot tests can catch unintended markup changes for stable components; integration tests exercise multiple components together (e.g. a full form flow) to verify they work correctly as a unit.

Q185. How would you handle errors gracefully in a component tree (Error Boundaries)?

A: Wrap sections of the tree in an Error Boundary component (a class component implementing `static getDerivedStateFromError`/`componentDidCatch`, since hooks don't yet support this) that catches rendering errors in its descendants and displays a fallback UI instead of crashing the whole app, optionally logging the error to a monitoring service.

Q186. ★What are common causes of memory leaks in a React Native app, and how do you catch them?

A: Common causes include missing `useEffect` cleanup for timers/subscriptions/listeners, holding references to unmounted components (e.g. calling `setState` after unmount), retaining large images/data in memory unnecessarily, and unclosed native resources like camera or location watchers; they're often caught via memory profiling tools (Xcode Instruments, Android Studio Profiler), console warnings, or noticing gradually degrading performance/RAM usage during extended use.

Q187. How would you approach migrating a class-component-heavy codebase to functional components and hooks?

A: Migrate incrementally rather than all at once — start with leaf/simple components, convert lifecycle methods to equivalent `useEffect` calls, replace `this.state`/`setState` with `useState`/`useReducer`, extract shared logic from HOCs/mixins into custom hooks, and rely on automated tests (or add them first) to verify behavior is preserved at each step.

Q188. What's your strategy for handling API errors and retries in a data-fetching layer?

A: Centralize fetching logic (e.g. in a service layer or via a library like React Query/SWR) so error handling, retry policies (with backoff), and timeout behavior are consistent; distinguish between recoverable errors (network blips — retry) and non-recoverable ones (4xx validation errors — surface to the user), and always show clear feedback and a manual retry option in the UI.

Q189. How would you implement optimistic UI updates, and what happens if the underlying request fails?

A: Update local state immediately to reflect the expected outcome (e.g. mark a todo as complete right away) before the network request resolves, then send the request in the background; if it fails, roll the state back to its previous value (or reconcile with the server's actual response) and show an error so the user knows the change didn't persist.

Q190. ★What are the 'Rules of Hooks', and why do hooks have to be called in the same order on every render?

A: The Rules of Hooks state that hooks must only be called at the top level of a function component (never inside conditions, loops, or nested functions) and only from React functions. This is because React tracks hooks by the order they're called in, matching each `useState`/`useEffect` call to its stored data by call order — calling them conditionally would shift that order and corrupt which stored state/effect each call maps to.

Q191. ★Why can't hooks be called conditionally or inside loops?

A: Because React relies on the call order of hooks within a component to associate each call with its internal storage slot; if a hook call is skipped or added conditionally between renders, the sequence shifts and React ends up matching the wrong stored state/effect to each hook call, causing bugs or crashes.

Q192. How would you code-review a pull request that introduces a new custom hook — what would you check for?

A: Check that it follows the Rules of Hooks, has a clear single responsibility and a descriptive `use`-prefixed name, correctly manages dependency arrays and cleanup for any effects, doesn't leak internal implementation details unnecessarily, is reasonably testable in isolation, and is documented/typed clearly enough for other developers to reuse it correctly.

Q193. How would you explain the render lifecycle of a functional component to a junior developer?

A: On each render, React calls the component function with its current props, which runs the function body (including hook calls) to produce JSX; React reconciles this against the previous output to determine what actually changed, commits those minimal changes to the DOM/native views, and then runs any effects whose dependencies changed, after the screen has been updated.

Q194. What trade-offs would you weigh when deciding whether a piece of state should live in a parent vs. be duplicated locally in a child?

A: Lifting state to a parent enables sharing/synchronizing it across siblings but can cause broader re-renders and adds indirection for state only one component actually cares about; keeping state local to a child is simpler and more encapsulated but can't easily be shared or observed by siblings — the right choice depends on whether more than one component genuinely needs that data.

Q195. How would you version and roll out an over-the-air (OTA) update to a released app?

A: Use an OTA update service (e.g. Expo Updates or CodePush) to publish a new JS bundle tied to a specific native binary/runtime version, roll it out gradually (e.g. to a percentage of users) to catch issues early, monitor

crash/error rates after release, and keep a rollback plan ready — noting that OTA updates can only ship JS/asset changes, not changes requiring new native code.

Q196. What's your approach to handling large images and media to keep list scrolling smooth?

A: Serve appropriately resized/compressed images from the backend or a CDN rather than full-resolution originals, use a caching image component (e.g. `expo-image` or `react-native-fast-image`) for efficient loading and memory reuse, lazy-load offscreen media, and avoid decoding large images synchronously on the main thread during scroll.

Q197. How would you enforce consistent code style and catch bugs early across a team (ESLint, Prettier, TypeScript strict mode)?

A: Configure ESLint (with React/React Native/hooks plugins) for catching bugs and enforcing conventions, Prettier for automatic consistent formatting, TypeScript in `strict` mode for stronger type safety, and wire them into pre-commit hooks (e.g. via Husky/lint-staged) and CI checks so violations are caught before merging rather than relying on manual review.

Q198. How would you handle deep linking so the app opens directly to a specific screen from a URL?

A: Configure the app's URL scheme/universal links (iOS) and intent filters (Android), define a linking configuration in React Navigation mapping URL paths to screens/params, and handle both cold-start deep links (app opened via a link) and links received while the app is already running, so navigation lands on the correct screen with the right params either way.

Q199. What's the difference between synchronous and asynchronous storage APIs, and why does that matter on app startup?

A: Synchronous storage APIs block the calling thread until the read/write completes, while asynchronous ones (like `AsyncStorage`) return a Promise and don't block; on app startup, blocking synchronous storage reads can delay the first render and make the app feel slow to launch, so most React Native storage APIs are deliberately asynchronous, requiring you to show a loading state while initial data loads.

Q200. How would you approach internationalization (i18n) in a React Native app?

A: Extract all user-facing strings into translation resource files keyed by locale, use an i18n library (e.g. `i18next`/`react-i18next` or `expo-localization` combined with `i18n-js`) to look up strings and handle pluralization/formatting, detect the device's locale (with a manual override option), and account for right-to-left layouts and locale-specific date/number formatting where relevant.